

Patient segmentation and hospital outcomes: adults aged 45-64

Vivienne Gunadhi | Synthetic EHR, 2018-2024 | Focused portfolio revision

Decision question

Among hospitalized adults aged 45-64, which baseline patient profiles are associated with different readmission, resource-use, and mortality patterns, and which segments merit further investigation for care-coordination planning?

This is an exploratory analysis of **synthetic, observational data**, not an intervention evaluation, clinical prediction system, or estimate of real-world treatment effects. Hospital charges are not costs or profit.

Analysis plan

1. Validate and join patient and outcome tables; define the age cohort upfront.
2. Describe disease burden and hospital outcomes within that cohort.
3. Retain focused observational comparisons and quantify uncertainty.
4. Reduce baseline clinical features with PCA; compare K-means solutions.
5. Profile clusters using outcomes excluded from model fitting.

The full patient population is used only for data-quality checks and cohort counts. Adults 65+ appear only as a contextual comparison for charges. Unrelated full-population EDA, exercise/gender/alcohol tests, seasonality, and duplicate clustering experiments have been removed.

1. Setup and input files

Place `patients.csv` and `patient_outcome.csv` in a `data/` folder next to this notebook, or change `DATA_DIR`.

Install dependencies if needed: `pip install pandas numpy scipy scikit-learn matplotlib seaborn nbformat`.

Execution status: raw CSV files were not supplied with the source notebook. Revised outputs are intentionally cleared; original cluster statistics must not be presented as results of this revised pipeline. Run all cells with the source CSVs before publishing findings.

```
from pathlib import Path
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

SEED = 3330
N_BOOT = 2000
AGE_MIN, AGE_MAX = 45, 64
DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)
sns.set_theme(style="whitegrid", palette=["#5275ad", "#8caadc", "#284570"])
pd.set_option("display.max_columns", 40)
```

```

paths = [DATA_DIR / "patients.csv", DATA_DIR / "patient_outcome.csv"]
missing = [str(p) for p in paths if not p.exists()]
if missing:
    raise FileNotFoundError("Add the original CSVs before running: " + ", ".join(missing))
patients = pd.read_csv(paths[0], dtype={"patient_id": "string"})
outcomes = pd.read_csv(paths[1], dtype={"patient_id": "string"})
patient_required = ["patient_id", "age", "bmi", "systolic_bp", "diastolic_bp", "heart_rate",
                    "charlson_index"]
outcome_required = ["patient_id", "admission_date", "discharge_date", "primary_drg",
                    "readmitted_30d", "in_hospital_death", "icu_admission",
                    "length_of_stay_days", "total_charges_usd"]
for name, frame, required in [("patients", patients, patient_required), ("outcomes", outcomes,
outcome_required)]:
    absent = sorted(set(required) - set(frame.columns))
    if absent:
        raise ValueError(f"{name}: missing columns {absent}")
    if frame["patient_id"].isna().any() or frame["patient_id"].duplicated().any():
        raise ValueError(f"{name}: patient_id must be complete and unique. Repeated admissions require a
different analysis unit.")

dx_cols = sorted(c for c in patients if c.startswith("dx_"))
if not dx_cols:
    raise ValueError("No diagnosis indicators (dx_ columns) found.")
for col in patient_required[1:] + dx_cols:
    patients[col] = pd.to_numeric(patients[col], errors="raise")
for col in ["readmitted_30d", "in_hospital_death", "icu_admission", "length_of_stay_days",
            "total_charges_usd"]:
    outcomes[col] = pd.to_numeric(outcomes[col], errors="raise")
for frame, cols in [(patients, dx_cols), (outcomes, ["readmitted_30d", "in_hospital_death",
            "icu_admission"])]:
    for col in cols:
        if not frame[col].dropna().isin([0, 1]).all():
            raise ValueError(f"{col} must contain only 0, 1 or missing values.")
for col in ["admission_date", "discharge_date"]:
    outcomes[col] = pd.to_datetime(outcomes[col], errors="raise")
if outcomes[["admission_date", "discharge_date"]].isna().any().any():
    raise ValueError("Admission/discharge dates must be complete.")
if (outcomes["discharge_date"] < outcomes["admission_date"]).any():
    raise ValueError("Discharge precedes admission.")
if not outcomes["admission_date"].dt.year.between(2018, 2024).all():
    raise ValueError("Admission dates fall outside the intended 2018–2024 window.")
if (outcomes["length_of_stay_days"].dropna() <= 0).any() or (outcomes["total_charges_usd"].dropna() <
0).any():
    raise ValueError("Review nonpositive stays or negative charges.")
display(pd.DataFrame({
    "table": ["patients", "outcomes"],
    "rows": [len(patients), len(outcomes)],
    "missing_cells": [patients.isna().sum().sum(), outcomes.isna().sum().sum()]
}))
# Missing days_to_readmission may be structural; it is not required for this analysis.

```

2. Cohort definition and focused feature engineering

Retain hospital records whose linked age is 45–64 inclusive. Age refers to the supplied patient-table age; the dataset must be checked to establish whether this represents age at admission.

A blood-pressure grouping reproduces the original project's numeric thresholds with the highest applicable band checked first. It is a descriptive measurement grouping, not a clinical diagnosis. Missing diagnoses are not silently counted as absence. DRG codes remain categorical and are used only to describe clusters.

```

patients["condition_count"] = patients[dx_cols].sum(axis=1, min_count=len(dx_cols))
# Exclude a recorded hypertension diagnosis from the comparison outcome to reduce circularity.
other_dx = [c for c in dx_cols if "hypertension" not in c.lower()]
patients["other_condition_count"] = patients[other_dx].sum(axis=1, min_count=len(other_dx))
s, d = patients["systolic_bp"], patients["diastolic_bp"]
patients["bp_group"] = np.select(
    [s.isna() | d.isna(), (s > 180) | (d > 120), (s >= 140) | (d >= 90),
     (s >= 130) | (d >= 80), (s >= 120)],
    ["Missing", "Highest band", "Stage 2 range", "Stage 1 range", "Elevated range"],
    default="Normal range"
)

```

```

)
joined = outcomes.merge(patients, on="patient_id", how="left", validate="one_to_one", indicator=True)
if not joined["_merge"].eq("both").all():
    raise ValueError("Some hospital records have no matching patient record.")
joined = joined.drop(columns="_merge")
if joined["age"].isna().any():
    raise ValueError("Age must be complete to define the cohort.")
adult = joined.loc[joined["age"].between(AGE_MIN, AGE_MAX)].copy().reset_index(drop=True)
senior = joined.loc[joined["age"] >= 65].copy()
if len(adult) < 20:
    raise ValueError("Too few cohort records for the proposed segmentation.")
adult["primary_drg"] = adult["primary_drg"].astype("string")
adult["charges_per_day"] = adult["total_charges_usd"] / adult["length_of_stay_days"]
display(pd.Series({
    "all_patient_records": len(patients),
    "all_hospital_records": len(joined),
    "hospitalized_45_64": len(adult),
    "share_of_hospital_records_pct": 100 * len(adult) / len(joined)
}, name="Cohort counts").to_frame())

outcome_cols = ["readmitted_30d", "in_hospital_death", "icu_admission",
    "length_of_stay_days", "total_charges_usd"]
display(adult[["age", "bmi", "charlson_index", "condition_count"] + outcome_cols].describe().T)
display(adult[outcome_cols].isna().sum().rename("missing_outcomes").to_frame())
prevalence = adult[dx_cols].mean().sort_values(ascending=False).mul(100)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
prevalence.sort_values().plot.barh(ax=axes[0], color="#5275ad")
axes[0].set(xlabel="Recorded prevalence (%)", ylabel="", title="Diagnosis burden: ages 45–64")
sns.histplot(adult["total_charges_usd"].dropna(), bins=25, ax=axes[1], color="#5275ad")
axes[1].set(xlabel="Hospital charges (USD)", title="Charges distribution")
plt.tight_layout()
plt.show()
# Denominators vary when an outcome is missing; retain outcome-specific counts.
display(adult[outcome_cols].agg(["count", "mean", "median"]).T)

```

3. Focused observational comparisons

These are **not randomized A/B tests**. Both comparisons are exploratory and two-sided.

- Within the target cohort: other diagnosis count for Stage 1-range versus normal-range measured BP.
- Context only: hospital charges for adults 45-64 versus adults 65+.

Report differences, uncertainty and sample sizes, rather than treating nonsignificance as proof of equivalence. Neither comparison adjusts for case mix or establishes causality.

```

def bootstrap_mean_difference(a, b, seed=SEED, n_boot=N_BOOT):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    a, b = a[np.isfinite(a)], b[np.isfinite(b)]
    if min(len(a), len(b)) < 2:
        raise ValueError("Each comparison group needs at least two valid observations.")
    rng = np.random.default_rng(seed)
    diffs = np.empty(n_boot)
    for i in range(n_boot):
        diffs[i] = rng.choice(a, len(a), replace=True).mean() - rng.choice(b, len(b),
            replace=True).mean()
    return a.mean() - b.mean(), *np.quantile(diffs, [0.025, 0.975])

comparisons = [
    ("Stage 1 minus normal BP: other diagnoses",
        adult.loc[adult["bp_group"].eq("Stage 1 range"), "other_condition_count"],
        adult.loc[adult["bp_group"].eq("Normal range"), "other_condition_count"]),
    ("Age 45–64 minus age 65+: charges USD",
        adult["total_charges_usd"], senior["total_charges_usd"])
]
test_rows = []
for label, a, b in comparisons:
    a, b = a.dropna(), b.dropna()
    if min(len(a), len(b)) < 2:
        print(f"Skipped {label}: insufficient observations.")
        continue
    difference, low, high = bootstrap_mean_difference(a, b)
    result = stats.ttest_ind(a, b, equal_var=False, alternative="two-sided")

```

```

test_rows.append(dict(comparison=label, n_a=len(a), n_b=len(b),
                     difference=difference, ci_low=low, ci_high=high, p_value=result.pvalue))
tests = pd.DataFrame(test_rows)
if not tests.empty:
    tests["p_bonferroni"] = (tests["p_value"] * len(tests)).clip(upper=1)
display(tests)
# Intervals are unadjusted exploratory percentile bootstrap intervals.

```

4. Outcome uncertainty and a clearly labeled planning scenario

Bootstrap at the **observed cohort size** to estimate uncertainty around observed rates and mean charges. Separately, simulate an assumed 5,500-patient cohort at the observed event probability. The latter is a conditional scenario interval, **not** a confidence interval for the observed rate and **not** a city-level or annual forecast.

Readmission uses all nonmissing recorded indicators. The source definition must establish whether deaths and other ineligible discharges are included. Rare mortality events require cautious interpretation.

```

def bootstrap_mean(values, seed=SEED, n_boot=N_BOOT):
    x = np.asarray(pd.Series(values).dropna(), dtype=float)
    if len(x) < 2:
        return np.nan, np.nan, np.nan
    rng = np.random.default_rng(seed)
    means = np.array([rng.choice(x, size=len(x), replace=True).mean() for _ in range(n_boot)])
    return x.mean(), *np.quantile(means, [0.025, 0.975])

uncertainty_rows = []
for i, col in enumerate(outcome_cols):
    estimate, low, high = bootstrap_mean(adult[col], seed=SEED+i)
    uncertainty_rows.append(dict(metric=col, n=adult[col].count(), estimate=estimate, ci_low=low,
                                ci_high=high))
uncertainty = pd.DataFrame(uncertainty_rows)
display(uncertainty)

ASSUMED_COHORT_SIZE = 5500
rng = np.random.default_rng(SEED)
scenario_rows = []
for col in ["readmitted_30d", "in_hospital_death"]:
    p = adult[col].mean()
    if pd.isna(p):
        continue
    simulated_counts = rng.binomial(ASSUMED_COHORT_SIZE, p, size=10000)
    lo, hi = np.quantile(simulated_counts, [0.025, 0.975])
    scenario_rows.append(dict(outcome=col, assumed_patients=ASSUMED_COHORT_SIZE,
                              observed_probability=p, expected_count=ASSUMED_COHORT_SIZE*p,
                              scenario_count_low=lo, scenario_count_high=hi))
display(pd.DataFrame(scenario_rows))
# This fixed-probability scenario excludes parameter uncertainty, temporal drift, and case-mix changes.

```

5. PCA on the target hospitalized cohort

Fit separate PCA blocks for continuous clinical measurements and binary diagnosis indicators. Retain 80% and 75% of variance respectively rather than silently resetting both blocks to four components.

Exclude hospital outcomes, DRG codes, and aggregate diagnosis counts from clustering. This avoids outcome leakage, artificial ordering of DRGs, and duplicating diagnosis information through summary scores. Lifestyle variables remain available for profiling but are not arbitrarily ordinal-encoded.

Use complete baseline records and explicitly report exclusions. Standardized PCA on binary diagnoses is exploratory: rare diagnoses can receive high weight, and alternative mixed-data methods are a future sensitivity analysis.

```

continuous_features = ["age", "bmi", "systolic_bp", "diastolic_bp", "heart_rate"]
baseline_features = continuous_features + dx_cols
complete = adult[baseline_features].replace([np.inf, -np.inf], np.nan).notna().all(axis=1)
cluster_base = adult.loc[complete].copy()
print(f"Clustering: {len(cluster_base):,} retained; {(~complete).sum():,} excluded for incomplete

```

```

baseline features.")
if len(cluster_base) < 20:
    raise ValueError("Too few complete baseline records.")

def fit_pca_block(frame, features, variance_target, prefix):
    varying = [c for c in features if frame[c].nunique() > 1]
    if not varying:
        raise ValueError(f"{prefix}: no varying features.")
    scaler = StandardScaler()
    scaled = scaler.fit_transform(frame[varying])
    pca = PCA(n_components=variance_target, svd_solver="full")
    scores = pca.fit_transform(scaled)
    names = [f"{prefix}_PC{i+1}" for i in range(scores.shape[1])]
    # Unit total variance per block: equal weight for clinical and diagnosis blocks.
    balanced_scores = scores / np.sqrt(pca.explained_variance_.sum())
    loadings = pd.DataFrame(pca.components_.T, index=varying, columns=names)
    print(f"{prefix}: {pca.n_components_} components, {pca.explained_variance_ratio_.sum():.1%} variance
    retained")
    display(loadings.round(3))
    return balanced_scores, pca, scaler, loadings

clinical_scores, clinical_pca, clinical_scaler, clinical_loadings = fit_pca_block(
    cluster_base, continuous_features, 0.80, "clinical")
disease_scores, disease_pca, disease_scaler, disease_loadings = fit_pca_block(
    cluster_base, dx_cols, 0.75, "diagnosis")
X = np.column_stack([clinical_scores, disease_scores])
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, model, title in zip(axes, [clinical_pca, disease_pca], ["Clinical measurements", "Diagnoses"]):
    ax.plot(range(1, model.n_components_+1), np.cumsum(model.explained_variance_ratio_), marker="o")
    ax.set(xlabel="Retained components", ylabel="Cumulative variance explained", title=title, ylim=(0,
    1.05))
plt.tight_layout()
plt.show()

```

6. K-means selection and stability

Compare $k=2-10$ using silhouette, minimum cluster size and repeat-initialization stability (adjusted Rand index). Select the highest-silhouette solution among those with at least 20 patients and 5% of the cohort per cluster. These minimum-size rules are explicit analyst choices, not clinical thresholds.

The original notebook overrode the silhouette winner with $k=3$. Here $k=3$ is a **sensitivity comparison only**, not an automatic choice. Weak silhouettes mean overlapping profiles, even when one k ranks highest. Repeated-seed stability measures optimization consistency, not external validity.

```

minimum_size = max(20, int(np.ceil(0.05 * len(X))))
models, rows = {}, []
for k in range(2, min(10, len(X)-1) + 1):
    model = KMeans(n_clusters=k, n_init=30, random_state=SEED).fit(X)
    labels = model.labels_
    if len(np.unique(labels)) < k:
        continue
    sil = silhouette_score(X, labels, sample_size=min(2000, len(X)), random_state=SEED)
    stability = []
    for offset in [1, 2, 3]:
        alternate = KMeans(n_clusters=k, n_init=10, random_state=SEED+offset).fit_predict(X)
        stability.append(adjusted_rand_score(labels, alternate))
    counts = np.bincount(labels)
    rows.append(dict(k=k, silhouette=sil, min_cluster_n=counts.min(),
                    mean_seed_ARI=np.mean(stability), inertia=model.inertia_,
                    eligible=counts.min() >= minimum_size))
    models[k] = model
selection = pd.DataFrame(rows).sort_values("k")
if selection.empty:
    raise ValueError("No valid clustering solution; inspect baseline feature variation.")
display(selection)
eligible = selection.loc[selection["eligible"]]
if eligible.empty:
    raise ValueError("No solution meets the declared minimum-size rule; review features before changing
    it.")
best_k = int(eligible.sort_values(["silhouette", "k"], ascending=[False, True]).iloc[0]["k"])
best_model = models[best_k]
print(f"Selected k={best_k}; minimum segment size rule={minimum_size}")

```

```

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(selection["k"], selection["silhouette"], marker="o")
ax.axvline(best_k, color="#284570", linestyle="--", label=f"Selected k={best_k}")
ax.set(xlabel="Number of clusters", ylabel="Silhouette score", title="Patient-profile separation")
ax.legend()
plt.show()
if 3 in models and best_k != 3:
    print("k=3 sensitivity: compare separation, size and seed stability in the table before preferring
    interpretability over fit.")
    display(pd.crosstab(best_model.labels_, models[3].labels_, rownames=["Selected solution"],
    colnames=["k=3 solution"]))

```

7. Patient profiles and held-out outcome descriptions

Outcomes are held out of feature construction, **not** held out as an independent validation sample. This is descriptive profiling of the same cohort. Cluster IDs have no intrinsic meaning and may change after rerunning or changing preprocessing.

Report event counts and denominators alongside rates. Wilson intervals avoid implying certainty when a cluster has zero observed events. Do not infer disease stage, progression, treatment effectiveness or medication recommendations from a diagnosis flag.

```

clustered = cluster_base.copy()
clustered["cluster"] = best_model.labels_
profile = clustered.groupby("cluster")[continuous_features + ["charlson_index",
    "condition_count"]].mean()
display(profile.round(2))
dx_profile = clustered.groupby("cluster")[dx_cols].mean().mul(100)
fig, ax = plt.subplots(figsize=(max(7, best_k*1.4), max(5, len(dx_cols)*0.36)))
sns.heatmap(dx_profile.T, annot=True, fmt=".1f", cmap="Blues", ax=ax)
ax.set(title="Recorded diagnosis prevalence (%) – hospitalized adults 45–64", xlabel="Cluster",
    ylabel="")
plt.tight_layout()
plt.show()
display(pd.crosstab(clustered["cluster"], clustered["primary_drg"],
    normalize="index").mul(100).round(1))
for col in ["smoking_status", "exercise_level", "insurance_type"]:
    if col in clustered:
        print(col)
        display(pd.crosstab(clustered["cluster"], clustered[col], normalize="index").round(3))

def wilson_interval(events, n, confidence=0.95):
    if n == 0:
        return np.nan, np.nan
    z = stats.norm.ppf(0.5 + confidence/2)
    p, denom = events/n, 1 + z*z/n
    center = (p + z*z/(2*n)) / denom
    half = z * np.sqrt(p*(1-p)/n + z*z/(4*n*n)) / denom
    return center-half, center+half

event_rows, resource_rows = [], []
for cluster, group in clustered.groupby("cluster"):
    for metric in ["readmitted_30d", "in_hospital_death", "icu_admission"]:
        observed = group[metric].dropna()
        events, n = int(observed.sum()), len(observed)
        low, high = wilson_interval(events, n)
        event_rows.append(dict(cluster=cluster, metric=metric, events=events, n=n,
            rate=events/n if n else np.nan, ci_low=low, ci_high=high))
    resource_rows.append(dict(cluster=cluster, n_patients=len(group),
        mean_charges=group["total_charges_usd"].mean(),
        median_charges=group["total_charges_usd"].median(),
        mean_stay_days=group["length_of_stay_days"].mean(),
        n_charges=group["total_charges_usd"].count(),
        n_stay=group["length_of_stay_days"].count()))
event_profile = pd.DataFrame(event_rows)
resource_profile = pd.DataFrame(resource_rows)
display(event_profile.round(4))
display(resource_profile.round(2))

readmit = event_profile.loc[event_profile["metric"].eq("readmitted_30d")].dropna(subset=["rate"])

```

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.errorbar(readmit["cluster"], readmit["rate"]*100,
            yerr=np.maximum(0, np.vstack([readmit["rate"]-readmit["ci_low"],
                                         readmit["ci_high"]-readmit["rate"]])*100),
            fmt="o", capsize=5, color="#5275ad")
ax.set(xlabel="Cluster", ylabel="Recorded readmission rate (%)", title="Readmission with 95% Wilson
      intervals")
ax.set_xticks(readmit["cluster"])
plt.show()
```

8. Evidence summary and portfolio-safe conclusions

The next cell generates descriptive statements only after the source data have been processed. Treat candidate segments as hypotheses for investigation, not clinically validated risk groups. Do not reuse original cluster names or percentages because the revised representation and selection rules change the model.

```
print(f"Analyzed {len(adult):,} hospitalized adults aged 45–64; {len(clustered):,} had complete
      clustering features.")
print(f"Selected {best_k} exploratory segments with silhouette {selection.loc[selection.k.eq(best_k),
      'silhouette'].iloc[0]:.3f}.")
if not readmit.empty:
    highest = readmit.loc[readmit["rate"].idxmax()]
    print(f"Cluster {int(highest['cluster'])} had the highest observed readmission rate: "
          f"{highest['rate']:.1%} ({int(highest['events'])}/{int(highest['n'])}); "
          f"95% Wilson interval {highest['ci_low']:.1%}–{highest['ci_high']:.1%}.")
for cluster in dx_profile.index:
    top = dx_profile.loc[cluster].nlargest(3)
    print(f"Cluster {cluster}, most prevalent recorded diagnoses: " +
          "; ".join(f"{name.removeprefix('dx_')}: {value:.1f}%" for name, value in top.items()))
print("These are synthetic-data associations, not evidence that an intervention reduces hospitalizations
      or improves survival.")

selection.to_csv(OUTPUT_DIR / "cluster_selection.csv", index=False)
profile.to_csv(OUTPUT_DIR / "patient_profiles.csv")
dx_profile.to_csv(OUTPUT_DIR / "diagnosis_prevalence_pct.csv")
event_profile.to_csv(OUTPUT_DIR / "cluster_events_and_intervals.csv", index=False)
resource_profile.to_csv(OUTPUT_DIR / "cluster_resource_use.csv", index=False)
uncertainty.to_csv(OUTPUT_DIR / "cohort_uncertainty.csv", index=False)
# Aggregate outputs only; no patient-level records are exported.
```

Interpretation and next steps

- **Care-coordination hypothesis:** review profiles with higher recorded readmission and diagnosis burden with domain experts; this analysis does not establish which intervention works.
- **Operational monitoring:** retain event denominators, charges and stay duration as distinct measures. Comparisons across time or providers require case-mix adjustment and consistent eligibility definitions.
- **Evidence limits:** synthetic generation mechanisms may create the apparent associations. Cluster separation can be weak; a forced segmentation is not necessarily clinically meaningful.
- **Validation needed:** assess sensitivity to diagnosis weighting, PCA thresholds, missingness and cohort definition; evaluate stability on independent or temporally separated data.
- **Not established:** longitudinal disease progression, prevented admissions, treatment benefits, causal effects, real-world ROI, or population-wide mortality/readmission forecasts.

Revision notes

Removed broad population tours and unrelated exploratory tests; corrected undefined-variable dependencies; replaced the missing sampling module with an inline bootstrap; separated bootstrap confidence intervals from fixed-probability planning scenarios; removed numerical DRG encoding and the forced k=3 override; retained explicit, target-cohort PCA fitting and result-dependent conclusions.

Original source: 2018-2024_Synthetic_EHR_Analysis_Vivienne_Gunadhi.ipynb. The original remains unchanged.